1. What is a Stored Procedure in MySQL?

A **Stored Procedure** is a block of SQL code saved in the database, which you can run anytime by calling it.

Think of it like a *function* in programming.

Why use stored procedures?

- Reuse the same SQL again and again
- Reduce code in applications
- Improve security (hide SQL logic)
- Better performance (precompiled)

2. Basic Syntax of a MySQL Stored Procedure

```
DELIMITER $$

CREATE PROCEDURE procedure_name()
BEGIN
  -- SQL statements
END $$

DELIMITER ;
```

Your FIRST Stored Procedure (Hello Example)

```
DELIMITER $$

CREATE PROCEDURE hello()
BEGIN
   SELECT 'Welcome to MySQL Stored Procedures!';
END $$

DELIMITER ;
```

Call it:

```
Call hello();
```

Procedure With Input Parameters

Parameters allow you to **send values** into the procedure.

Example: Display a student name

```
DELIMITER $$

CREATE PROCEDURE greet_student(IN student_name VARCHAR(50))
BEGIN
   SELECT CONCAT('Hello ', student_name, '!') AS message;
END $$

DELIMITER ;
```

Call it:

```
CALL greet_student('Rahul');
```

Procedure With Multiple Parameters

Example: Add 2 numbers:

```
DELIMITER $$

CREATE PROCEDURE add_numbers(IN a INT, IN b INT)
BEGIN
   SELECT a + b AS result;
END $$

DELIMITER ;
```

Call:

```
CALL add_numbers(10, 20);
```

Procedure With OUTPUT Parameter

Used to return values back to the caller.

```
DELIMITER $$

CREATE PROCEDURE square(IN num INT, OUT result INT)
BEGIN
   SET result = num * num;
END $$
```

```
DELIMITER ;
```

Call:

```
call square(20, @result);

select @result;
```

Procedure Using Variables, SET, IF

```
DELIMITER $$

CREATE PROCEDURE check_even_odd(IN num INT)
BEGIN
   IF (num % 2 = 0) THEN
      SELECT 'Even' AS result;
   ELSE
      SELECT 'Odd' AS result;
   END IF;
END $$

DELIMITER ;
```

Call:

```
CALL check_even_odd(7);
```

Procedure With SELECT and INSERT

Let’s assume you have a table:

```
CREATE TABLE students (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(50),
   marks INT
);
```

Procedure to add new student:

```
DELIMITER $$

CREATE PROCEDURE add_student(IN s_name VARCHAR(50), IN s_marks
INT)
BEGIN
```

```
    INSERT INTO students(name, marks) VALUES(s_name, s_marks);
END $$

DELIMITER ;
```

Call:

```
CALL add_student('Riya', 88);
```

Procedure Using UPDATE

```
DELIMITER $$

CREATE PROCEDURE update_marks(IN s_id INT, IN new_marks INT)
BEGIN
    UPDATE students
    SET marks = new_marks
    WHERE id = s_id;
END $$

DELIMITER ;
```

Call:

```
CALL update_marks(1, 95);
```

Procedure Using DELETE

```
DELIMITER $$

CREATE PROCEDURE delete_student(IN s_id INT)
BEGIN
    DELETE FROM students WHERE id = s_id;
END $$

DELIMITER ;
```

Call:

```
CALL delete_student(2);
```

What is a CURSOR in MySQL?

A **cursor** allows you to fetch rows **one-by-one** from a query result.

Normally:

```
SELECT * FROM students;
```

returns all rows at once.

But sometimes you need to process each row individually, like:

- Calculate tax for each employee
- Print each student name one-by-one
- Update records row by row
- Move data from one table to another

For this, MySQL gives us **cursors**.

Basic Structure of a MySQL CURSOR

A cursor has **4 main parts**:

1. Declare variable

2. Declare cursor

3. Declare NOT FOUND handler

4. Open cursor → Fetch → Loop → Close cursor

General format:

```
DECLARE done INT DEFAULT FALSE;
DECLARE cur_name CURSOR FOR
  SELECT columns FROM table;
```

```
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
        OPEN cur_name;
        read_loop: LOOP
            FETCH cur_name INTO variable1, variable2, ...;
            IF done THEN
                LEAVE read_loop;
            END IF;
            -- use the row data here
        END LOOP;
CLOSE cur_name;
```

Your FIRST CURSOR EXAMPLE

Goal: Fetch all student names one-by-one and display them.

```
DELIMITER $$

  DECLARE done INT DEFAULT FALSE;
  DECLARE s_name VARCHAR(50);
  DECLARE cur_student CURSOR FOR
     SELECT name FROM students;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
  OPEN cur_student;
```

```
    read_loop: LOOP
        FETCH cur_student INTO s_name;
        IF done THEN
            LEAVE read_loop;
        END IF;
        SELECT s_name AS student_name;
    END LOOP;
    CLOSE cur_student;
END $$
DELIMITER ;
```

Call the procedure:

```
CALL cursor_example1();
```

Cursor Fetching MULTIPLE Columns

Goal: Fetch student name + marks together.

```
DELIMITER $$
CREATE PROCEDURE cursor_example2()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE s_name VARCHAR(50);
    DECLARE s_marks INT;

    DECLARE cur CURSOR FOR
        SELECT name, marks FROM students;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO s_name, s_marks;
        IF done THEN
            LEAVE read_loop;
        END IF;
        SELECT s_name AS Name, s_marks AS Marks;
    END LOOP;
    CLOSE cur;
END $$
DELIMITER ;
```

Call:

```
CALL cursor_example2();
```

Cursor With Calculations

Goal: For each student, print grade based on marks.

```
DELIMITER $$


CREATE PROCEDURE cursor_example3()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE s_name VARCHAR(50);
    DECLARE s_marks INT;
```

```
DECLARE s_grade VARCHAR(10);

DECLARE cur CURSOR FOR
    SELECT name, marks FROM students;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN cur;

read_loop: LOOP
    FETCH cur INTO s_name, s_marks;
    IF done THEN
        LEAVE read_loop;
    END IF;
    IF s_marks >= 90 THEN
        SET s_grade = 'A';
    ELSEIF s_marks >= 75 THEN
        SET s_grade = 'B';
    ELSE
        SET s_grade = 'C';
    END IF;
    SELECT s_name AS Name, s_marks AS Marks, s_grade AS Grade;
END LOOP;
CLOSE cur;
```

```
END $$

DELIMITER ;
```

Call:

```
CALL cursor_example3();
```

Cursor With UPDATE

Goal:

Increase marks of each student by 5.

```
DELIMITER $$


CREATE PROCEDURE cursor_update_example()

BEGIN

   DECLARE done INT DEFAULT FALSE;

   DECLARE s_id INT;

   DECLARE s_marks INT;


   DECLARE cur CURSOR FOR

      SELECT id, marks FROM students;


   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;


   OPEN cur;
```

```
    read_loop: LOOP
        FETCH cur INTO s_id, s_marks;
        IF done THEN
            LEAVE read_loop;
        END IF;
        UPDATE students
        SET marks = s_marks + 5
        WHERE id = s_id;
    END LOOP;
    CLOSE cur;
END $$
DELIMITER ;
```

Call:

```
CALL cursor_update_example();
```

Cursor With INSERT (Transfer data to new table)

Create new table:

```
CREATE TABLE passed_students (
    name VARCHAR(50),
    marks INT
);
```

Procedure:

```
DELIMITER $$
```

```
CREATE PROCEDURE cursor_insert_example()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE s_name VARCHAR(50);
    DECLARE s_marks INT;

    DECLARE cur CURSOR FOR
        SELECT name, marks FROM students WHERE marks >= 70;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO s_name, s_marks;
        IF done THEN LEAVE read_loop; END IF;

        INSERT INTO passed_students(name, marks)
        VALUES (s_name, s_marks);
    END LOOP;
    CLOSE cur;
END $$
DELIMITER ;
```

Call:

```
CALL cursor_insert_example();
```

What is a Stored Function?

A **Stored Function**:

Takes input
Performs some calculation
Returns exactly one value
Can be used inside **SELECT**, **WHERE**, **ORDER BY**, etc. (unlike procedures)

CREATE FUNCTION function_name(parameters)

RETURNS datatype

BEGIN

-- statements

RETURN value;

END;

Enable Function Creation (IMPORTANT)

Some MySQL setups block functions for security.

Run this 1-time:

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

Your FIRST Stored Function

Return square of a number

```
DELIMITER $$


CREATE FUNCTION square(n INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN n * n;
END $$
DELIMITER ;
```

Call:

```
SELECT square(5);
```

Output → 25

Stored Function With Multiple Parameters

Add two numbers

```
DELIMITER $$
CREATE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN a + b;
END $$
DELIMITER ;
```

Use:

```
SELECT add_numbers(10, 20);
```

Stored Function With IF Conditions

Return grade based on marks

```
DELIMITER $$


CREATE FUNCTION get_grade(marks INT)

RETURNS VARCHAR(5)

DETERMINISTIC

BEGIN

    IF marks >= 90 THEN

        RETURN 'A';

    ELSEIF marks >= 75 THEN

        RETURN 'B';

    ELSE

        RETURN 'C';

    END IF;

END $$

DELIMITER ;
```

Use:

```
SELECT name, marks, get_grade(marks) AS grade FROM students;
```

Stored Function That Accesses a Table

Let's say you have:

```
students(id, name, marks)
```

Return marks of a student by ID

```
DELIMITER $$

CREATE FUNCTION get_marks(sid INT)

RETURNS INT

DETERMINISTIC

BEGIN

    DECLARE m INT;


    SELECT marks INTO m

    FROM students

    WHERE id = sid;


    RETURN m;

END $$

DELIMITER ;
```

Use:

```
SELECT get_marks(2);
```

Function With Calculations

Calculate tax for salary

```
DELIMITER $$


CREATE FUNCTION calc_tax(salary DECIMAL(10,2))

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

    RETURN salary * 0.18;  -- 18% GST example

END $$

DELIMITER ;
```

Use:

```
SELECT calc_tax(10000);
```

Using Stored Functions in Queries

Assume your table:

```
employees(emp_id, name, salary)
```

Use function to calculate annual salary

Function:

```
DELIMITER $$
```

```
CREATE FUNCTION yearly_salary(monthly DECIMAL(10,2))

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

    RETURN monthly * 12;

END $$



DELIMITER ;
```

Use inside SELECT:

```
SELECT name, salary, yearly_salary(salary) AS Annual_Salary

FROM employees;
```

Database Trigger

A **Database Trigger** is a **special kind of stored program** that automatically runs when a specific **event** happens in a table.

Think of a trigger like an **automatic reaction**.

Something happens in the table →
Trigger runs automatically

A **database trigger** is a piece of code that executes automatically when an **INSERT**, **UPDATE**, or **DELETE** operation occurs on a table.

Why are triggers used?

Triggers help in tasks like:

Automatically maintaining logs (audit)

- Who updated a record?
- When was data inserted?

Enforcing business rules

- Prevent deletion of important data
- Prevent negative salary updates

Validating data before saving

- Check if age is valid
- Check stock quantity before sale

Maintaining consistency between tables

- Update total marks table when marks table changes
- Update product stock when new order is inserted

Automating tasks

- Auto-fill fields
- Auto-calculate values

Events that cause a trigger to fire

Trigger can be executed:

BEFORE INSERT

AFTER INSERT

BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE

Each one fires at a different stage.

Basic Syntax (MySQL)

```
CREATE TRIGGER trigger_name

BEFORE|AFTER INSERT|UPDATE|DELETE

ON table_name

FOR EACH ROW

BEGIN

    -- Trigger logic here

END;
```

Understanding OLD and NEW

Inside triggers, MySQL gives two special records:

NEW.column_name

- Represents the *new values* being inserted or updated

OLD.column_name

- Represents the *existing values* before update or delete

Examples:

- INSERT trigger → only **NEW** available
- DELETE trigger → only **OLD** available
- UPDATE trigger → both available (OLD and NEW)

Example 1 — BEFORE INSERT Trigger

Automatically convert name to uppercase before inserting.

```
CREATE TRIGGER before_student_insert

BEFORE INSERT ON students

FOR EACH ROW

BEGIN

    SET NEW.name = UPPER(NEW.name);

END;
```

Now:

```
INSERT INTO students(name, marks) VALUES ('riya', 90);
```

Actual stored value → **RIYA**

Example 2 — AFTER INSERT Trigger (Audit Log)

Create log table:

```
CREATE TABLE student_log (

    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    student_name VARCHAR(50),
    action VARCHAR(20),
    action_time DATETIME
);
```

Trigger:

```
CREATE TRIGGER after_student_insert
AFTER INSERT ON students
FOR EACH ROW
BEGIN
    INSERT INTO student_log(student_name, action, action_time)
    VALUES (NEW.name, 'INSERT', NOW());
END;
```

Whenever you insert a new student, the log table gets an entry automatically.